

Teste de Software

Refatoração de Testes e Detecção de Test Smells

Aluno: João Pedro Campos

Matrícula: 777543

Instituição: PUC Minas Lourdes

Disciplina: Teste de Software

1. Análise de Smells

A suíte original `userService.smelly.test.js` passava nos testes, mas apresentava problemas de clareza, manutenibilidade e confiabilidade. O objetivo da refatoração foi manter o comportamento validado, melhorando a estrutura dos cenários de teste e eliminando os principais test smells apontados pela ferramenta.

- Eager Test: um mesmo teste criava e validava vários usuários ao mesmo tempo. Isso dificultava identificar qual regra falhou e misturava responsabilidades diferentes dentro de um único cenário.
- Lógica condicional no teste: havia uso de `if/else` dentro do teste para decidir o resultado esperado. Esse padrão torna o teste mais difícil de ler e esconde os caminhos de execução. O ESLint confirmou esse problema com a regra `jest/no-conditional-expect`.
- Fragile Test: o teste de relatório dependia de uma linha formatada exatamente, incluindo ID, nome, status e quebra de linha. Pequenas mudanças de formatação poderiam quebrar o teste sem alteração real no comportamento importante.
- Disabled Test: foi identificado um teste desabilitado, indicado pela regra `jest/no-disabled-tests`. Testes desabilitados reduzem a confiança na suíte, pois deixam comportamentos sem verificação ativa.

2. Processo de Refatoração

A refatoração foi feita em um novo arquivo, `__tests__/userService.clean.test.js`, preservando o arquivo original para comparação. A nova versão organiza os testes pelo padrão Arrange, Act, Assert, usa nomes mais descritivos e separa comportamentos diferentes em testes independentes.

Antes: teste com loop, if e expects condicionais

```
for (const user of todosOsUsuarios) {  
  const resultado = userService.deactivateUser(user.id);  
  if (!user.isAdmin) {  
    expect(resultado).toBe(true);  
    expect(usuarioAtualizado.status).toBe('inativo');  
  } else {  
    expect(resultado).toBe(false);  
  }  
}
```

No trecho acima, a validação depende de uma condição interna. Isso faz com que o teste tenha múltiplos caminhos de execução e dificulta a leitura do comportamento esperado.

Depois: cenários separados e objetivos

```

test('desativa um usuário comum', () => {
  // Arrange
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

  // Act
  const resultado = userService.deactivateUser(usuarioComum.id);
  const usuarioAtualizado = userService.getUserById(usuarioComum.id);

  // Assert
  expect(resultado).toBe(true);
  expect(usuarioAtualizado.status).toBe('inativo');
});

test('não desativa um usuário administrador', () => {
  // Arrange
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  // Act
  const resultado = userService.deactivateUser(usuarioAdmin.id);
  const usuarioAtualizado = userService.getUserById(usuarioAdmin.id);

  // Assert
  expect(resultado).toBe(false);
  expect(usuarioAtualizado.status).toBe('ativo');
});

```

Com essa separação, cada teste valida uma regra específica. Quando ocorre uma falha, fica mais fácil identificar exatamente qual comportamento foi quebrado.

3. Depois da Refatoração

A principal decisão foi transformar um teste com várias responsabilidades em testes pequenos, focados e independentes. Em vez de um teste decidir internamente o que verificar, cada regra ganhou um cenário próprio.

- A lógica condicional foi removida dos asserts.
- Os nomes dos testes passaram a explicar o comportamento esperado.
- O teste de exceção foi reescrito com `expect(fn).toThrow(...)`, garantindo falha quando a exceção esperada não é lançada.
- O teste de relatório passou a verificar informações relevantes, como nome e status, sem depender de uma linha formatada de forma rígida.
- A suíte continuou passando, o que indica que o comportamento foi preservado após a melhoria estrutural.

Exemplo de teste limpo criado no arquivo refatorado

```
test('cria um usuário maior de idade com status ativo', () => {
  // Arrange
  const dadosUsuario = {
    nome: 'Fulano de Tal',
    email: 'fulano@teste.com',
    idade: 25,
  };

  // Act
  const usuarioCriado = userService.createUser(
    dadosUsuario.nome,
    dadosUsuario.email,
    dadosUsuario.idade
  );

  // Assert
  expect(usuarioCriado).toEqual(expect.objectContaining({
    nome: dadosUsuario.nome,
    email: dadosUsuario.email,
    idade: dadosUsuario.idade,
    status: 'ativo',
  }));
});
```

4. Relatório da Ferramenta

O ESLint foi configurado com `eslint-plugin-jest` e executado com `npx eslint ..`. A ferramenta apontou problemas no arquivo original, principalmente expects condicionais e teste desabilitado. Em seguida, o arquivo limpo foi validado individualmente e não apresentou erros de lint.

Resultado no arquivo original

```
test/userService.smelly.test.js
44:9   error    Avoid calling `expect` conditionally  jest/no-conditional-expect
46:9   error    Avoid calling `expect` conditionally  jest/no-conditional-expect
49:9   error    Avoid calling `expect` conditionally  jest/no-conditional-expect
73:7   error    Avoid calling `expect` conditionally  jest/no-conditional-expect
77:3   warning   Disabled test                        jest/no-disabled-tests
77:3   warning   Test has no assertions                jest/expect-expect

6 problems (4 errors, 2 warnings)
```

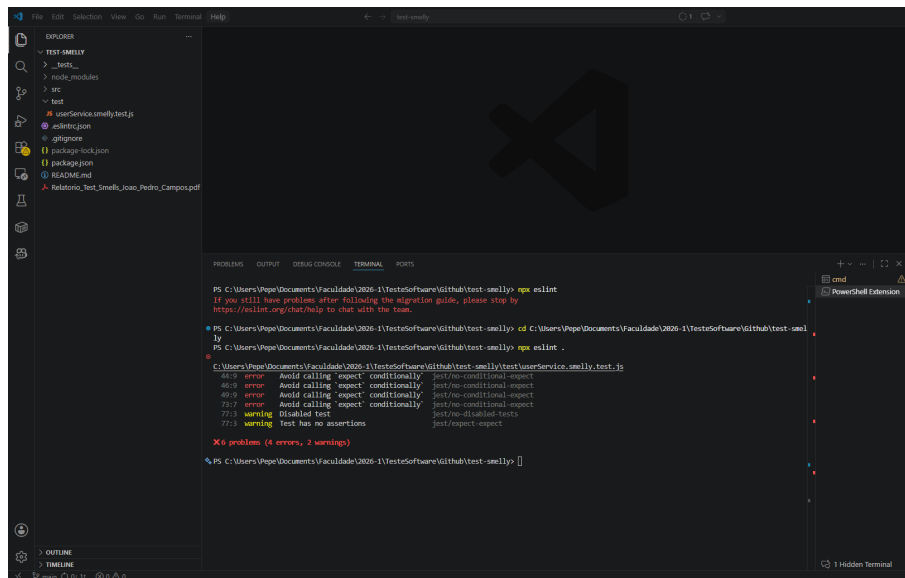
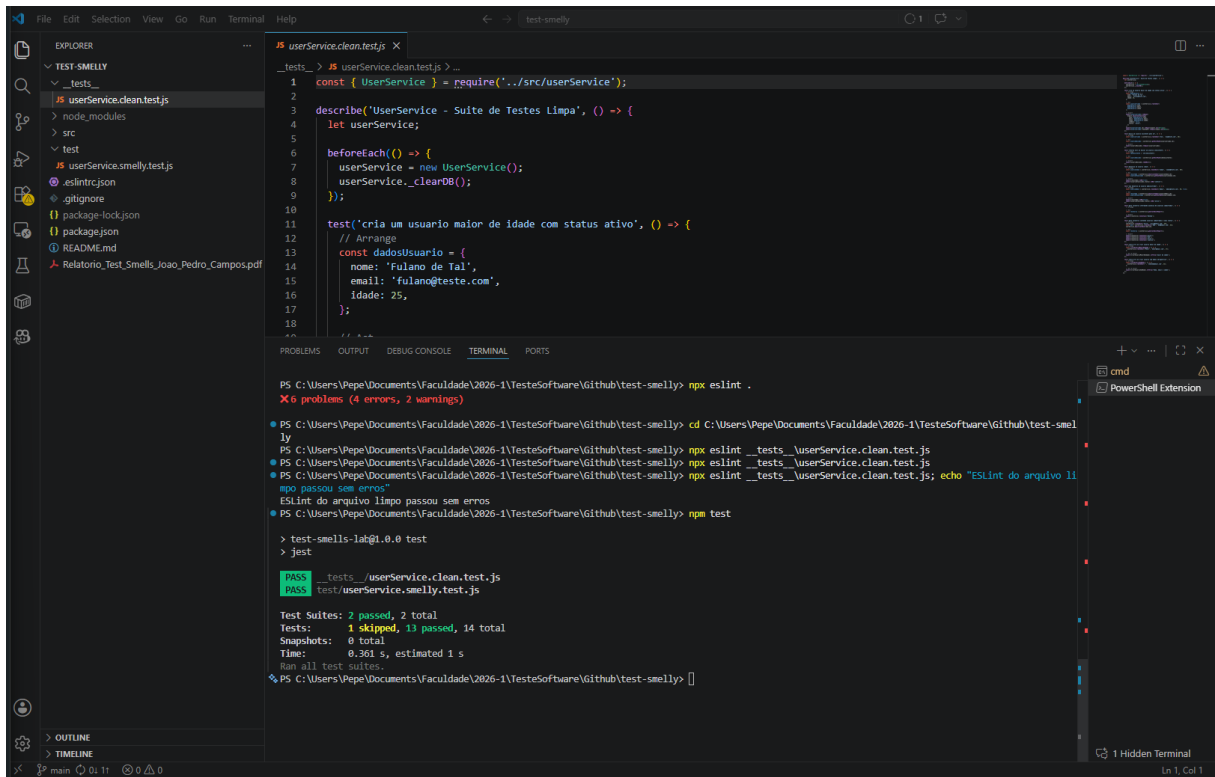


Figura 1 - Execução do ESLint mostrando os problemas encontrados no teste original.

5. Evidências de Validação

Após a refatoração, o arquivo limpo foi verificado separadamente com ESLint e a suíte de testes foi executada com npm test. O resultado mostra que o arquivo refatorado passou sem erros de lint e que as duas suítes de teste foram executadas com sucesso.



The screenshot shows the VS Code interface with the Explorer sidebar on the left displaying the file structure of a project named 'test-smelly'. The main editor area shows the file 'userService.clean.test.js' with the following code:

```
1 const { UserService } = require('../src/userService');
2
3 describe('UserService - Suite de Testes Limpa', () => {
4   let userService;
5
6   beforeEach(() => {
7     userService = new UserService();
8     userService._clearDB();
9   });
10
11   test('cria um usuario maior de idade com status ativo', () => {
12     // Arrange
13     const dadosUsuario = {
14       nome: 'Fulano de Tal',
15       email: 'fulano@teste.com',
16       idade: 25,
17     };
18   });
19 });
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Users\Pepe\Documents\Faculdade\2026-1\TesteSoftware\Github\test-smelly> npx eslint .
Xs problems (4 errors, 2 warnings)

• PS C:\Users\Pepe\Documents\Faculdade\2026-1\TesteSoftware\Github\test-smelly> cd C:\Users\Pepe\Documents\Faculdade\2026-1\TesteSoftware\Github\test-smelly
• PS C:\Users\Pepe\Documents\Faculdade\2026-1\TesteSoftware\Github\test-smelly> npx eslint __tests__\userService.clean.test.js
• PS C:\Users\Pepe\Documents\Faculdade\2026-1\TesteSoftware\Github\test-smelly> npx eslint __tests__\userService.clean.test.js
• PS C:\Users\Pepe\Documents\Faculdade\2026-1\TesteSoftware\Github\test-smelly> npx eslint __tests__\userService.clean.test.js; echo "ESLint do arquivo limpo passou sem erros"
ESLint do arquivo limpo passou sem erros
• PS C:\Users\Pepe\Documents\Faculdade\2026-1\TesteSoftware\Github\test-smelly> npm test

> test-smells-lab@1.0.0 test
> jest

PASS __tests__\userService.clean.test.js
PASS test\userService.smelly.test.js

Test Suites: 2 passed, 2 total
Tests: 1 skipped, 13 passed, 14 total
Snapshots: 0 total
Time: 0.361 s, estimated 1 s
Ran all test suites.
```

Figura 2 - ESLint no arquivo refatorado e execução do npm test com as suítes aprovadas.

Resumo da validação

- ESLint do arquivo limpo: passou sem erros.
- Test Suites: 2 passed, 2 total.
- Tests: 1 skipped, 13 passed, 14 total.
- A suíte original foi mantida para comparação, e a nova suíte demonstra a versão refatorada.

6. Conclusão

A escrita de testes limpos contribui para a sustentabilidade do projeto. Testes pequenos, focados e organizados pelo padrão Arrange, Act, Assert são mais fáceis de entender, manter e evoluir. A utilização do ESLint ajudou a transformar boas práticas em verificação automática, acelerando a identificação de problemas recorrentes e aumentando a confiança na suíte de testes.

Repositório: <https://github.com/Campos001/test-smelly>

Arquivo refatorado: __tests__/userService.clean.test.js